

Wie die Verschlüsselung funktioniert

Ein kompakter Einstieg in die drei Bausteine hinter diesem Bericht — damit die folgende Analyse auch ohne Krypto-Vorwissen lesbar ist. Bewusst konzeptionell gehalten.

Ein Tresor im Tresor

Die Vertraulichkeit ruht auf **zwei** voneinander unabhängigen Schichten. Außen liegt das gewohnte **TLS** (das „https“ der Transportverschlüsselung). Darin liegt der eigentliche Inhalt **noch einmal** verschlüsselt — eine anwendungseigene Schicht, die man **Application-Level Encryption (ALE)** nennt. Wer nur die äußere Schicht öffnet, findet innen weiterhin einen verschlossenen Kasten. Der Klartext entsteht erst *innerhalb* der App.



Folge: Ein klassisches Mitschneiden des Netzwerkverkehrs (TLS-MITM) liefert nur den inneren, verschlüsselten Container — nicht den Messwert.

Zwei Verfahren: Geheimhaltung und Echtheit

AES-GCM — verschlüsseln (Vertraulichkeit *und* Integrität)

Ein **symmetrisches** Verfahren: derselbe Schlüssel ver- und entschlüsselt. „GCM“ liefert neben dem Ciphertext ein 16 Byte langes **Siegel** (Auth-Tag). Beim Entschlüsseln wird das Siegel nachgerechnet — passt es nicht exakt zu Schlüssel, Nonce und Daten, **bricht die Operation ab und liefert nichts**. Manipulation ist damit ausgeschlossen. Eine kurze Einmal-Zahl (die **Nonce**, 12 Byte) sorgt dafür, dass dieselbe Nachricht nie zweimal gleich aussieht.

Klartext + Nonce (12 B) + Schlüssel → **AES-GCM** → Ciphertext + Auth-Tag (16 B)

Entschlüsseln ist der umgekehrte Weg; stimmt das Tag nicht, gibt es keinen Klartext.

RSA-PSS — signieren (Echtheit)

Ein **asymmetrisches** Verfahren mit Schlüsselpaar: der **private** Schlüssel *signiert*, der **öffentliche** *prüft*. Signiert wird nicht die ganze Nachricht, sondern ihr Hash — plus einem Zufalls-Salt („PSS“), weshalb jede Signatur anders aussieht, obwohl sie dieselbe Nachricht bestätigt. RSA-PSS **verschlüsselt nichts**; es beweist nur, dass eine Nachricht echt und unverändert ist. AES sorgt für Geheimhaltung, RSA für Authentizität.

White-Box-Kryptografie — der Tresor, der selbst auf- und zuschließt

Eine **White-Box** (Fachname „Secure Key Box“) rechnet mit Schlüsseln, ohne sie je als lesbaren Wert offenzulegen — der Schlüssel ist in die Rechenlogik „eingebacken“. Das ist die zentrale Hürde: Man

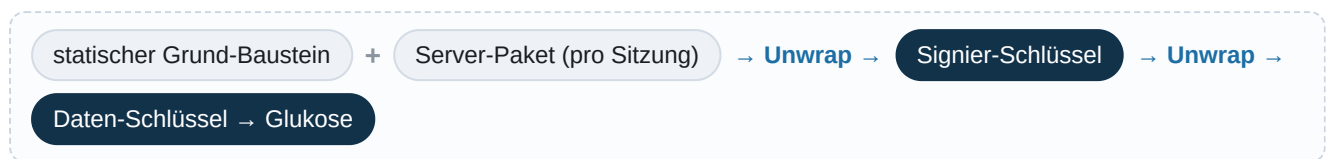
kann ihn nicht einfach aus dem Speicher fischen, weil er dort nie als zusammenhängender Wert auftaucht.

Die zentrale Erkenntnis: Schlüssel werden *ausgepackt*, nicht *ausgehandelt*

Intuitiv würde man annehmen, der Sitzungsschlüssel werde bei jedem Start frisch *ausgehandelt* (etwa per Diffie-Hellman). Für den laufenden Betrieb stimmt das nicht. Tatsächlich entstehen die Sitzungsschlüssel **deterministisch** aus zwei Zutaten:

- einem **statischen Grund-Baustein**, der fest in der Krypto-Komponente eingebettet ist, und
- einem **pro-Sitzung gelieferten Paket** von der Serverseite.

Aus beidem wird der Schlüssel in einer **zweistufigen Leiter** entpackt (Key-Unwrap): Stufe 1 ergibt einen **Signier-Schlüssel**, Stufe 2 den **Daten-Schlüssel**, der die eigentlichen Nutzdaten — auch den Glukosewert — entschlüsselt. „Der Schlüssel“ ist also weder ein einzelner statischer Wert noch ein frei ausgehandeltes Geheimnis, sondern das Ergebnis eines deterministischen Entpackens.



Mit diesem Bild im Kopf lohnt sich der Blick in die Details. Der folgende Bericht zeigt, wie sich diese Kette trotz eines aggressiven Manipulationsschutzes analysieren ließ — und wie sie sich am Ende sogar in einem eigenen Prozess vollständig reproduzieren ließ.

LibreLinkUp Application-Level-Encryption — ein Reverse-Engineering-Writeup

Technischer Erfahrungsbericht über die Krypto- und Schutz-Architektur der LibreLinkUp-Android-App und die Wege, den eigenen Glukosewert für die eigene Smartwatch nutzbar zu machen.

Status: Ziel erreicht (zwei unabhängige Verfahren, eines davon offline verifiziert) — durchgehend auf einem **ungerooteten** Gerät. **Fokus:** Verständnis & Methodik. Diese öffentliche Fassung ist bewusst ein *Erkenntnis-Bericht*, kein Kochrezept.

⚠ Medizinischer Haftungsausschluss. Dies ist ein technischer Sicherheits- und Reverse-Engineering-Bericht, **kein medizinisches Dokument**. Die beschriebenen Verfahren und sämtliche damit gewonnenen Glukosewerte (CGM-Daten) dürfen **unter keinen Umständen** für medizinische Entscheidungen herangezogen werden — insbesondere nicht für die Dosierung von Insulin oder andere Therapieanpassungen. Selbst erstellte Auslese-Wege können fehlerhafte, verzögerte, veraltete oder unvollständige Werte liefern. Für Therapieentscheidungen sind ausschließlich die vom Hersteller zugelassenen, geprüften Anzeigergeräte und -Apps sowie ärztlicher Rat maßgeblich.

Scope, Ethik & bewusste Auslassungen

Das beschriebene Vorgehen betraf ausschließlich die **eigene** App-Installation, die **eigenen** Zugangsdaten und die **eigenen** Gesundheitsdaten auf einem Gerät im eigenen Besitz. Motivation war persönliche Interoperabilität: den eigenen Wert auf der eigenen Uhr sehen.

Wichtig war mir dabei, das gesamte Vorgehen auf einem **ungerooteten** Gerät im Alltagszustand durchzuführen — ohne Root, ohne Custom-ROM, ohne Aushebeln der Plattform-Sicherheit. Das hält das Gerät in seinem normalen, vertrauenswürdigen Zustand (kein Garantieverlust, keine geschwächte Android-Sicherheit) und entspricht dem realistischen Rahmen eines gewöhnlichen Nutzers. Zugleich ist es die **höhere Hürde**: Viele gängige Hook- und Instrumentierungs-Ansätze setzen Root voraus — der hier beschriebene Weg kommt bewusst ohne aus.

Diese Veröffentlichung ist als **Security-/RE-Research-Writeup** gedacht. Sie erklärt Architektur, Konzepte und Methodik, damit andere das *Verständnis* nachvollziehen können. Sie enthält **absichtlich nicht**:

- lauffähige Exploit-/Hook-Quelltexte,
- die konkreten Smali-Patches, die den Manipulationsschutz der App neutralisieren,
- build-spezifische Speicheradressen (Offsets) und Krypto-Konstanten,
- Schlüsselmaterial oder personenbezogene Daten jeglicher Art.

Diese Bausteine würden den Text in ein schlüsselfertiges Werkzeug zur Umgehung des Schutzes einer fremden App verwandeln — das ist nicht das Ziel. Wer denselben Weg für seine eigenen Daten geht, muss die gerätespezifischen Details selbst erarbeiten; genau das ist Teil der Legitimitäts-Schwelle.

1. Das Problem in einem Satz

LibreLinkUp verschlüsselt seine API-Nutzlast **doppelt**: einmal per TLS (der normale HTTPS-Transport) und darüber hinaus noch einmal **innerhalb der App** (Application-Level Encryption, „ALE“). Selbst wer TLS aufbricht, sieht deshalb nur einen verschlüsselten Container — nicht den Glukosewert.

2. Systemüberblick

Baustein	Rolle
Die App (<code>org.nativescript.LibreLinkUp</code>)	„Follower“-App; App-Logik in JavaScript (NativeScript), nativer Unterbau.
Eine native Krypto-Bridge (in Go geschrieben)	kapselt die eigentliche Verschlüsselung; enthält eine White-Box-Krypto (Secure Key Box).
Eine native Schutz-Bibliothek (RASP)	<i>Runtime Application Self-Protection</i> — erkennt Manipulation/Debugger/Root und lässt die App gezielt abstürzen.
Kommerzieller Obfuskator (DexGuard)	verschleiert den Java/Smali-Code und bringt die Integritätsprüfungen mit.

Kernpunkt: Die Vertraulichkeit hängt **nicht** an TLS, sondern an der ALE-Schicht, und die wiederum an der White-Box in der Go-Bridge.

3. Die ALE-Schicht

Jeder verschlüsselte Body hat konzeptionell die Form:

```
{ "data": "<IV>.<Ciphertext>.<Auth-Tag>", // base64, punktgetrennt
  "signature": "<RSA-Signatur über data>" }
```

- **AES-GCM-128** liefert Vertraulichkeit *und* Integrität (12-Byte-Nonce, 16-Byte-Auth-Tag).
- **RSA-PSS** signiert den `data` -Teil (Authentizität).

Zur Einordnung: Die **punktgetrennte Zusammensetzung** `IV.Ciphertext.Auth-Tag` ist eine **app-spezifische Verpackung** von LibreLinkUp, **kein** kryptografisches Paradigma. AES-GCM erzeugt standardmäßig nur Ciphertext und einen Auth-Tag; wie diese Bestandteile zusammen mit dem IV/der Nonce serialisiert werden — hier Base64 mit Punkt als Trennzeichen —, ist eine reine Design-Entscheidung der App und variiert von Implementierung zu Implementierung.

Ein klassischer TLS-Man-in-the-Middle liefert damit nur diesen Container — der Grund, warum ein On-Device-Ansatz nötig ist.

4. Die Krypto-Bridge existiert erst zur Laufzeit

Die Bridge liegt in der APK nicht als normale, lesbare Bibliothek, sondern **gepackt/verschlüsselt**. Erst beim Start entpackt ein nativer Loader sie in eine **temporäre** Datei im privaten App-Ordner, lädt sie und **löscht sie sofort wieder** (der offene Datei-Deskriptor hält sie am Leben). Konsequenzen:

1. Die APK-Kopie ist für die Analyse wertlos — der echte Code existiert nur im **entschlüsselten Laufzeit-Zustand**.
2. Das Zeitfenster ist winzig, die Datei wächst progressiv und verschwindet dann → man muss den Schreibvorgang „berennen“, um ein vollständiges Abbild zu bekommen.
3. Genau über dieses entschlüsselte Abbild bildet der Manipulationsschutz zur Laufzeit die **Prüfsumme** — an diesem nativen Code darf ein Hook deshalb nichts verändern.

Aus dem vollständigen Abbild lassen sich die exportierten Krypto-Funktionen über die dynamische Symboltabelle des ELF ablesen (die Debug-Symbole sind entfernt, die *dynamischen* Exporte müssen

aber vorhanden bleiben, sonst wäre die Bibliothek nicht nutzbar).

5. Die Schutzschichten (warum es schwierig ist)

1. **Isolierter Linker-Namespace** — Standard-Symbolsuche aus einer anderen Bibliothek findet die Funktionen nicht; Adressen müssen über die Modul-Basis aus `/proc/<pid>/maps` bestimmt werden.
2. **Native Integritätsprüfung** — über den Code der Bridge wird zur Laufzeit eine Prüfsumme gebildet; jeder Byte-Patch an diesem Code fällt auf.
3. **Java-Ebene-RASP** in den Lifecycle-Methoden — die wirksamste Schutzschicht: bei erkannter Manipulation reagiert die App bewusst destabilisierend (zufällige Ausnahmen auf mehreren Threads, eine Warn-Meldung, Speicher-Allokationsbomben) und beendet sich **vor** dem Login.
4. **Dynamisch registrierte JNI-Methoden** der Schutz-Bibliothek — man kann sie nicht durch einen Dummy ersetzen.
5. **Frida-Erkennung** — bekannte Instrumentierungs-Signaturen werden im Speicher entdeckt.

Merksatz: Stabil bleibt die App nur, wenn man **beide** Ebenen adressiert — den nativen Code *nicht* verändern **und** die Java-RASP entschärfen.

6. Verfahren 1 — am Klartext abgreifen (Live-Hook)

Egal wie gut der Schlüssel geschützt ist: Die Entschlüsselungsfunktion **muss** irgendwann den Klartext in einen Ausgabepuffer schreiben. Wer diesen Puffer nach dem Aufruf liest, hat den Wert — ohne den Schlüssel zu kennen.

Der erste, naive Weg (den Funktionsanfang mit einem Sprung überschreiben) funktionierte zwar — lieferte echten Klartext — wurde aber von der Integritätsprüfung erkannt und führte zum Absturz. Er verändert eben Bytes im geschützten Code.

Der **stabile** Weg nutzt **Hardware-Breakpoints**: die Debug-Register der CPU lösen aus, *bevor* eine Instruktion läuft, **ohne ein Byte am geschützten Krypto-Code zu ändern**. Die Prüfsumme über diesen Code sieht ihn „sauber“. Weil ein Prozess sich nicht selbst so beobachten darf, übernimmt ein winziger mitgestarteter Beobachter-Prozess das Setzen der Breakpoints und liest nach dem Aufruf den entschlüsselten Puffer aus dem Speicher der App.

Zur Klarstellung (kein Widerspruch): Die App selbst wird sehr wohl verändert — die Beobachter-Bibliothek wird über eine Bytecode-Änderung (Smali-Patch) in die App geladen, und weitere Patches entschärfen die Selbstschutz-Prüfungen. Der Punkt der Hardware-Breakpoints ist enger: Nur der **prüfsummen-geschützte native Krypto-Code** bleibt unangetastet — genau der Teil, den die Integritätsprüfung überwacht.

Erkenntnis: Gegen Laufzeit-Integritätsprüfungen schlägt ein *byte-erhaltender* Beobachtungs-Hook (Hardware-Breakpoint) den klassischen *byte-verändernden* Inline-Hook.

7. Verfahren 2 — die White-Box als Dienstleister (Standalone-Orakel)

Statt in der laufenden App abzugreifen, kann man die entschlüsselte Krypto-Bridge in einen **eigenen** Prozess laden und die Krypto-Funktionen selbst aufrufen — als „Orakel“. Der Schlüssel muss dafür nie extrahiert werden.

Die Hürde: Die internen Krypto-Objekte sind lebende Objekte mit eigenen Heap-Puffern — man kann sie nicht einfach aus einem Speicherabzug kopieren. Die Lösung ist, sie **nicht zu kopieren, sondern von der echten Engine im eigenen Prozess neu erzeugen zu lassen**, indem man die exakte Initialisierungsreihenfolge der App nachspielt. (Nennen wir es „Replay-Mint“.)

Warum das im Fremdprozess überhaupt trägt. Der entscheidende technische Hebel: Die Krypto-Bridge ist in Go geschrieben (via cgo) und bringt eine **eigene Laufzeitumgebung** mit. Diese **initialisiert sich beim Laden der Bibliothek (dlopen) selbständig** — dieselben Konstruktoren, die beim Start der App laufen, laufen unverändert auch im eigenen Prozess. Deshalb ist das „Nachspielen“ kein blinder Versuch: Nach dem Laden steht die Engine bereits einsatzbereit da, und die statische Initialisierung, auf der alle weiteren Objekte aufbauen, hat sich selbst erledigt. Ohne diese Selbst-Initialisierung der Runtime wäre der gesamte Orakel-Ansatz aussichtslos.

Dieses Verfahren wurde **byte-identisch verifiziert**: Der im eigenen Prozess rekonstruierte Sitzungsschlüssel entschlüsselt eine echte Server-Antwort exakt. Das beweist eine wichtige, oft unterschätzte Eigenschaft: **die White-Box ist nicht prozessgebunden** — der Sitzungsschlüssel ergibt sich deterministisch aus in der Bibliothek eingebetteten Daten plus einem pro-Sitzung vom Server gelieferten Bundle. Die **Geräte-Portabilität** folgt aus der Account-Unabhängigkeit dieser eingebetteten Daten, wurde aber nicht über mehrere Geräte hinweg verifiziert (Befund auf einem Gerät).

Methodische Einordnung — was hier wirklich passiert. Dieser Angriff **bricht die White-Box nicht**: Er greift die mathematische Schlüssel-Verschleierung an keiner Stelle an (kein DCA/DFA, keine Schlüssel-Extraktion). Es handelt sich vielmehr um klassisches **„Code Lifting“ bzw. API-Missbrauch** — die White-Box wird als **Black-Box-Dienstleister im eigenen Prozess** weiterbetrieben und exakt so aufgerufen, wie die App es täte. Der Rohschlüssel bleibt die gesamte Zeit im geschützten Inneren der White-Box; er wird **umgangen, nicht geknackt**. Diese Unterscheidung ist wichtig: Die Härtung der White-Box-Mathematik ist damit nicht widerlegt — sie ist nur an der falschen Stelle wirksam, wenn die Aufruf-Schnittstelle im eigenen Prozess reproduzierbar ist.

8. Das Schlüssel-Modell (die zentrale Erkenntnis)

Das intuitive Modell — „der AES-Schlüssel wird pro Sitzung per Diffie-Hellman (ECDH) neu ausgehandelt“ — **stimmt für den laufenden Betrieb nicht**. ECDH gehört zur einmaligen Erst-Einrichtung (Provisionierung), nicht zum minütlichen Abruf.

Tatsächlich gibt es eine **zweistufige Schlüssel-Leiter** (zweimaliges „Key-Unwrap“) und **drei Schlüsselrollen**:

Rolle	Herkunft (konzeptionell)	Wofür
Bootstrap-Schlüssel	direkt aus einem statischen , in die Bibliothek eingebetteten Datenobjekt	ent-/verschlüsselt große statische Konfigurations-Blobs und die Request-Bodies
Signier-Schlüssel	Stufe 1 der Key-Leiter (aus einem pro-Sitzung-Server-Bundle)	signiert Requests
Daten-Schlüssel	Stufe 2 der Key-Leiter	entschlüsselt die eigentlichen Nutzdaten — u. a. den Glukosewert

Für den reinen Glukose-Klartext genügt die zweite Stufe. „Der Schlüssel“ ist also nicht *ein* statischer Wert und auch kein frei ausgehandeltes Geheimnis, sondern das Ergebnis eines deterministischen Entpackens: **statisches Bibliotheks-Material + pro-Sitzung-Bundle → Sitzungsschlüssel**.

Woher dieses Modell stammt (Methodik). Die Schlüssel-Leiter ist nicht gesetzt, sondern **aus einem Laufzeit-Trace hergeleitet**. Über einen Hardware-Breakpoint auf dem Eintritt der Unwrap- und Entschlüsselungs-Funktionen wurden bei jedem Aufruf die Argumente gemäß der ARM64-Aufrufkonvention ausgelesen (**AAPCS64**: die ersten Argumente stehen in festgelegten CPU-Registern, weitere auf dem Stack — man weiß also, in welchem Register welcher Parameter liegt). Drei Beobachtungen führten zum Modell:

1. **Zwei Stufen** — die Unwrap-Funktion feuerte pro Login **zweimal** mit unterschiedlichem Eingabematerial; die Ausgabe des ersten Laufs speiste erkennbar den zweiten.
2. **Statisch vs. pro-Sitzung** — die einen Eingabe-Blobs blieben über alle Logins und Geräte **bit-identisch** (sie sind fest in die Bibliothek eingebettet), die anderen **variieren bei jedem Login** (das sind die server-gelieferten Bundles). Genau dieser Unterschied trennt Bootstrap-Material von Sitzungs-Material.
3. **Rollenzuordnung** — welcher entpackte Schlüssel welche Rolle hat, wurde über die **Objekt-Identität** bestätigt (welcher Schlüssel anschließend welche Operation speist — Signieren vs. Entschlüsseln) und über den **Inhalt des jeweils erzeugten Klartexts**.

Damit ist die Leiter kein axiomatisches Statement, sondern ein am Laufzeitverhalten belegter Befund.

9. Übergreifende Erkenntnisse

- **Doppelte Verschlüsselung (ALE über TLS)** macht reinen Netzwerk-MITM wirkungslos; der interessante Punkt liegt dort, wo der Klartext ohnehin entsteht.
- **Laufzeit-Integritätsprüfungen schlägt man nicht durch cleverere Byte-Patches am geschützten Code, sondern durch reine Beobachtung ohne Byte-Änderung** an eben diesem Code (Hardware-Breakpoints) — während die App an anderer Stelle (per Bytecode-Patch) durchaus modifiziert wird.
- **Mehrschichtiger Schutz erfordert mehrschichtige Antworten**: nativer Code *und* Java-RASP müssen zusammen betrachtet werden.
- **White-Box heißt nicht automatisch prozessgebunden**: hier ließ sich die Kette deterministisch offline in einem fremden Prozess reproduzieren — der eigentlich überraschende Befund. Die Geräte-Portabilität folgt aus der Account-Unabhängigkeit der eingebetteten Blobs, ist aber nur auf einem Gerät belegt.

10. Alternative Wege (Einordnung)

Weg	RE-Aufwand	robust gegen erzwungene ALE	Bewertung
REST-Client (offene Follower-Schnittstelle)	keiner	nein (bricht, sobald ALE serverseitig erzwungen wird)	billigster Weg, solange er hält
On-Device-Hook (Verfahren 1)	mittel	ja	liefert genau das, was die echte App sieht
White-Box-Orakel (Verfahren 2)	hoch	ja	patch-frei, überlebt ALE-Zwang; die sauberste Eigenlösung

Weg	RE-Aufwand	robust gegen erzwungene ALE	Bewertung
Schlüssel-Extraktion (DCA/DFA gegen die White-Box)	sehr hoch	ja	unwirtschaftlich — das Orakel erreicht dasselbe, ohne den Schlüssel zu sehen
Sensor direkt lesen (Jugluco/xDrip u. ä.)	gering–mittel	ja (kein Server)	technisch stark, wenn man den Sensor selbst trägt — entkoppelt aber von LibreLinkUp/LibreView (Nachteil, wenn der behandelnde Arzt den Follower-Dienst nutzt)

11. Datenschutz

Alle entschlüsselten Daten und alles Schlüsselmateriale sind personenbezogene Gesundheitsdaten und bleiben lokal auf dem eigenen Gerät. In dieser Veröffentlichung kommen weder echte Schlüssel, UUIDs oder E-Mail-Adressen noch build-spezifische Adressen vor.

Dieser Bericht dokumentiert eine persönliche Interoperabilitäts-Lösung und die dabei gewonnenen Erkenntnisse über App-Härtung und White-Box-Kryptografie. Er ist kein Aufruf und keine Anleitung, fremde Daten oder fremde Geräte anzugreifen.